- Text color
- Button styling

Practice 5: Flexbox

Create navbar:

- Logo left
- Menu center/right
- Use justify-content

Practice 6: Grid

Create product grid:

- 3 columns desktop
- 1 column mobile

Practice 7: Responsive Design

Make any page mobile-friendly using media query.

Practice 8

Build from scratch:

Mini Website

Include:

- Header
- Navbar
- Hero section
- About section
- Services (cards)
- Contact section
- Footer

Must use:

- Flexbox
- Grid
- Media Queries
- Semantic tags

Day 15

JavaScript Introduction + Variables

1. What is JavaScript?

JavaScript is a **programming language used to make websites interactive**.

HTML = Structure

CSS = Design

JavaScript = Logic + Actions

Simple Meaning: JavaScript = Website behavior control language

2. Why JavaScript is Important?

Without JavaScript:

- Website is static
- No actions
- No interactivity

With JavaScript:

- Buttons work
- Forms validate
- Dynamic content
- Games, apps, animations

Real Examples:

- Login validation (wrong password alert)
- Cart update in eCommerce
- Live chat
- Popups
- Sliders

3. Where JavaScript Runs?

JavaScript runs in:

Browser (Chrome, Edge, Firefox)

Example:

Open browser → Right click → Inspect → Console

4. First JavaScript Program

```
<html>
<body>
<script>
  console.log("Hello JavaScript!");
</script>
</body>
</html>
```

Output:

Displayed in console:

Hello JavaScript!

5. What is a Variable?

A variable is a **container to store data**.

Simple Meaning:

Variable = Box to store values

Example:

let name = "Alagu";

Here:

- name = variable
- "Vetri" = value

6. Types of Variables in JS

JavaScript has 3 ways to declare variables:

1. var

var age = 25;

Problems:

- Can be redeclared
- Not safe for large projects

2. let (modern ⭐)

let name = "John";

Features:

Can change value

- Cannot redeclare

let age = 20;

age = 25; // allowed

3. const (constant)

const country = "India";

Features:

- Cannot change value
- Fixed data

const pi = 3.14;

// pi = 3.15

7. Variable Comparison Table

Type	Reassign	Redeclare	Use
var	Yes	Yes	Avoid
let	Yes	No	Best choice
const	No	No	Fixed values

8. Data Types (Basic Intro)

Variables store different types of data:

String

let name = "HTML";

Number

```
let age = 25;
```

Boolean

```
let isStudent = true;
```

Null

```
let value = null;
```

Undefined

```
let x;
```

9. Example Program (Variables)

```
<html>
<body>
<script>
let name = "Alagu";
let age = 25;
console.log("Name: " + name);
console.log("Age: " + age);
</script>
</body>
</html>
```

Tasks (Day 15 Practice)

Task 1

Create a variable for your name and print it

Task 2

Create a variable for your age

Task 3

Store your city name in a variable

Task 4

Use let and change value

Task 5

Use const for country name

Task 6

Print 3 variables using console.log

Task 7

Create variables for student details (name, age, course)

Task 8

Try var, let, const and observe differences

Task 9

Create a simple profile using variables

Task 10 (Challenge)

Create a "User Info System":

- name
- age
- email

Print all values in console

Mini Projects

Project 1: Profile Logger

- Store name, age, city
- Print in console

Project 2: Student Info System

- Name, class, marks
- Display using console.log

Project 3: Product Info

- Product name, price, brand
- Store in variables

Project 4: Simple Form Data Simulation

- Name, email, password (variables)
- Print all values

Day 16: JavaScript Operators + Conditions (Detailed Explanation)

1. What are Operators in JavaScript?

Operators are symbols used to **perform operations on values or variables**.

Simple Meaning:

Operators = Doing calculations or comparisons

Example:

```
let a = 10;
let b = 5;
console.log(a + b);
```

2. Types of Operators

JavaScript has **5 main types**:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Conditional (Ternary) Operator

3. Arithmetic Operators

Used for mathematical operations.

Operators:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (remainder)

Example:

```
let a = 10;
let b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.33
console.log(a % b); // 1
```

Real Use Case:

- Shopping cart total
- Discount calculation
- Age calculation

4. Assignment Operators

Used to assign values.

Operators:

Operator	Meaning
=	assign
+=	add and assign
-=	subtract and assign

Example:

```
let x = 10;
x += 5; // x = x + 5
console.log(x); // 15
```

6.Comparison Operators

Used to compare values.

Operators:

Operator	Meaning
==	equal (value only)
===	strict equal (value + type)
!=	not equal
>	greater than
<	less than
>=	greater or equal
<=	less or equal

Example:

```
let a = 10;
let b = 5;
console.log(a > b); // true
console.log(a == b); // false
console.log(a === "10"); // false
```

Important:

- == → only value
- === → value + type (BEST PRACTICE ⭐)

6. Logical Operators

Used to combine conditions.

Operators:

Operator	Meaning
&&	AND
!	NOT

Example:

```
let age = 20;
console.log(age > 18 && age < 30); // true
console.log(age > 18 || age < 10); // true
console.log(!(age > 18)); // false
```

Real Use Case:

- Login system
- Form validation
- Access control

7. Conditions in JavaScript

Conditions are used to **make decisions in code**.

1. if statement

```
let age = 18;
if (age >= 18) {
   console.log("You are eligible to vote");
}
```

2. if...else

```
let age = 16;
if (age >= 18) {
   console.log("Allowed");
} else {
   console.log("Not allowed");
}
```

3. else if ladder

```
let marks = 75;

if (marks >= 90) {
   console.log("A Grade");
} else if (marks >= 70) {
   console.log("B Grade");
} else {
   console.log("C Grade");
}
```

8. Nested if

```
let age = 20;
let hasID = true;

if (age >= 18) {
   if (hasID) {
      console.log("Entry allowed");
   }
}
```

9. Ternary Operator (Short Condition)

Shortcut for if-else

Syntax:

```
condition ? true : false;
```

Example:

```
let age = 20;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result);
```

10. Full Example (All Concepts)

```
<html>
<body>
<script>
let age = 20;
let marks = 80;
// condition
if (age >= 18 && marks > 50) {
   console.log("Eligible Student");
} else {
   console.log("Not Eligible");
}
</script>
</body>
</html>
```

Tasks (Day 16 Practice)

Task 1

Add two numbers using arithmetic operators

Task 2

Check greater number between two values

Task 3

Use modulus operator

Task 4

Use AND operator in condition

Task 5

Check if age is eligible for voting

Task 6

Create grading system using marks

Task 7

Check even or odd number

Task 8

Use else-if ladder for traffic signal system

Task 9

Use ternary operator for age check

Task 10 (Challenge)

Create login system:

- username
- password

If correct → "Login success"

Else → "Login failed"

Mini Projects

Project 1: Age Checker System

- Check voting eligibility

Project 2: Student Grade System

- Marks → Grade (A/B/C)

Project 3: Calculator Logic

- Add, subtract, multiply using operators

Project 4: Simple Login Validator

- Username + password check

Day 17: JavaScript Loops (Detailed Explanation)

1. What is a Loop?

A loop is used to **repeat a block of code multiple times** until a condition becomes false.

Simple Meaning:

Loop = Doing same work again and again automatically

Example:

Instead of writing:

```
console.log("Hello");
console.log("Hello");
console.log("Hello");
```

We use loop:

```
for (let i = 1; i <= 3; i++) {
   console.log("Hello");
}
```

2. Why Loops are Important?

Loops are used in real projects for:

- Showing product lists
- Displaying user data
- Processing arrays
- Automation tasks

3. Types of Loops in JavaScript

1. for loop
2. while loop
3. do...while loop
4. for...of loop (advanced intro)

4. for Loop

Syntax:

```
for (initialization; condition; increment) {
  // code block
}
```

Example:

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Output:

1
2
3
4
5

Explanation:

- let i = 1 → start
- i <= 5 → condition
- i++ → increase

Real Use Case:

- Displaying product cards
- Showing menu items
- Looping through data

5. while Loop

Condition is checked first

Syntax:

```
while (condition) {
  // code
}
```

Example:

```
let i = 1;
while (i <= 5) {
  console.log(i);
```

```
    i++;
}
```

Output:

1 2 3 4 5

Important:

- If condition is wrong → infinite loop

6. do...while Loop

Runs at least ONCE

Syntax:

```
do {
   // code
} while (condition);
```

Example:

```
let i = 1;
do {
   console.log(i);
   i++;
} while (i <= 5);
```

Key Difference:

- Executes first
- Then checks condition

7. for...of Loop (Intro)

Used for arrays.

Example:

```
let fruits = ["apple", "banana", "mango"];

for (let fruit of fruits) {
   console.log(fruit);
}
```

Output:

apple
banana
mango

8. Loop Comparison Table

Loop	Best Use
for	Known number of iterations
while	Unknown condition

Loop	Best Use
do...while	Must run at least once
for...of	Arrays

9. Common Mistakes

Forgetting increment (i++)
Infinite loop
Wrong condition
Using loop when not needed

10. Nested Loop (Basic Intro)

Loop inside loop

Example:

```
for (let i = 1; i <= 3; i++) {
   for (let j = 1; j <= 2; j++) {
      console.log(i, j);
   }
}
```

Use Cases:

- Tables
- Grid systems
- Calendar layouts

Tasks (Day 17 Practice)

Task 1

Print numbers 1 to 10 using for loop

Task 2

Print even numbers from 1 to 20

Task 3

Print reverse numbers from 10 to 1

Task 4

Use while loop to print 1–5

Task 5

Use do-while loop

Task 6

Print multiplication table of 5

Task 7

Loop through array of 5 names

Task 8

Find sum of numbers from 1 to 10

Task 9

Print all odd numbers

Task 10 (Challenge)

Create login attempt system:

- Allow only 3 attempts using loop
- If correct → success message
- Else → lock system

Mini Projects

Project 1: Number Counter App

- Count 1 to 100 using loop

Project 2: Product List Display

- Loop through product array
- Display items

Project 3: Multiplication Table Generator

- Input number → show table

Project 4: Login Attempt System

- Max 3 attempts using loop

Day 18: JavaScript Functions (Detailed Explanation)

What is a Function?

A function is a **block of reusable code** that performs a specific task.

Simple Meaning:

Function = A reusable machine that does a job when called

Example:

Instead of writing same code again and again:

```
console.log("Hello");
console.log("Hello");
console.log("Hello");
```

We create a function:

```
function greet() {
   console.log("Hello");
}
greet();
greet();
greet();
```

Why Functions are Important?

Functions help to:

- Reuse code

- Avoid repetition
- Organize code neatly
- Build large applications easily

Function Structure

Syntax:

```
function functionName() {
   // code block
}
```

Example:

```
function sayHello() {
   console.log("Hello World");
}
```

Calling Function:

```
sayHello();
```

Function Types in JavaScript

1. Simple Function
2. Function with Parameters
3. Function with Return Value
4. Anonymous Function
5. Arrow Function (modern ⭐)

Function with Parameters

Parameters = input values

Syntax:

```
function add(a, b) {
   console.log(a + b);
}
```

Example:

```
add(10, 5);
```

Output:

15

Meaning:

- a, b → inputs
- function works with different values

6. Function with Return Value

Return gives output back

Syntax:

```
function multiply(a, b) {
   return a * b;
}
```

Example:

```
let result = multiply(4, 5);
console.log(result);
```

Output:

20

Why return is important?

- Used in calculations
- Used in real apps (shopping cart, login, etc.)

Anonymous Function

☞ Function without name

Example:

```
let greet = function() {
   console.log("Hello");
};
greet();
```

Use Case:

- Event handling
- Temporary functions

Arrow Function (MODERN ★)

Short syntax for functions

Syntax:

```
let add = (a, b) => {
   return a + b;
};
```

Short version:

```
let add = (a, b) => a + b;
```

Example:

```
console.log(add(10, 20));
```

Function Types Comparison

Type	Use
Simple function	Basic tasks
Parameters	Input-based tasks
Return	Output-based logic

Type	Use
Anonymous	Temporary functions
Arrow	Modern short syntax

Function Flow Example

```
function login(user, pass) {
   if (user === "admin" && pass === "1234") {
      return "Login Success";
   } else {
      return "Login Failed";
   }
}
console.log(login("admin", "1234"));
```

Tasks (Day 18 Practice)

Task 1

Create a function that prints your name

Task 2

Function to add two numbers

Task 3

Function to subtract numbers

Task 4

Function to check even or odd

Task 5

Function with 3 parameters (sum)

Task 6

Function that returns square of number

Task 7

Create arrow function for multiplication

Task 8

Function for simple calculator (+ - * /)

Task 9

Function to check voting eligibility

Task 10 (Challenge)

Create login system using function:

- username + password check
- return success/fail message

Mini Projects

Project 1: Calculator Functions

- add, subtract, multiply, divide functions

Project 2: User Validation System

- login function
- password check

Project 3: Grade Calculator

- function returns grade based on marks

Project 4: Shopping Cart Total

- function calculates total price

Day 19

JavaScript Arrays (Detailed Explanation)

What is an Array?

An array is a **special variable that stores multiple values in one place**.

Simple Meaning:

Array = One box → many items inside

Example:

Instead of this

```
let student1 = "A";
let student2 = "B";
let student3 = "C";
```

We use array ✓

```
let students = ["A", "B", "C"];
```

Why Arrays are Important?

Arrays are used for:

- Storing product lists
- User data
- Menu items
- API data
- Large datasets

Creating an Array

Syntax:

```
let arrayName = [value1, value2, value3];
```

Example:

```
let fruits = ["Apple", "Banana", "Mango"];
```

4. Array Index

Each item has a position called **index**

Index starts from 0

Example:

```
let fruits = ["Apple", "Banana", "Mango"];
```

Index	Value
0	Apple
1	Banana
2	Mango

Accessing values:

```
console.log(fruits[0]); // Apple
console.log(fruits[2]); // Mango
```

Updating Array Values

```
let fruits = ["Apple", "Banana", "Mango"];
fruits[1] = "Orange";
console.log(fruits);
```

Output

```
["Apple", "Orange", "Mango"]
```

Array Methods

push() → Add at end

```
let fruits = ["Apple", "Banana"];
fruits.push("Mango");
console.log(fruits);
```

pop() → Remove last item

```
fruits.pop();
```

shift() → Remove first item

```
fruits.shift();
```

unshift() → Add at beginning

```
fruits.unshift("Grapes");
```

length → Count items

```
console.log(fruits.length);
```

Looping Through Arrays

for loop:

```
let fruits = ["Apple", "Banana", "Mango"];
for (let i = 0; i < fruits.length; i++) {
   console.log(fruits[i]);
}
```

for...of loop:

```
for (let fruit of fruits) {
   console.log(fruit);
}
```

Array with Functions (REAL USE ⭐)

```
function printFruits(fruits) {
   for (let fruit of fruits) {
      console.log(fruit);
   }
}
printFruits(["Apple", "Banana", "Mango"]);
```

Real Life Analogy

Array Concept	Real Life
Array	Shopping bag
Items	Products
Index	Item position

Important Array Features

- Stores multiple values
- Index-based system
- Dynamic (can grow/shrink)
- Used in real applications

Tasks (Day 19 Practice)

Task 1

Create array of 5 fruits

Task 2

Print first and last element

Task 3

Update second element

Task 4

Add new item using push()

Task 5

Remove last item using pop()

Task 6

Loop array using for loop

Task 7

Loop array using for...of

Task 8

Find length of array

Task 9

Create array of 5 student names and print them

Task 10 (Challenge)

Create product list system:

- Store 5 products
- Add new product
- Remove one product
- Print final list

Mini Projects

Project 1: Product List Manager

- Add/remove products
- Display list

Project 2: Student List System

- Store student names
- Loop and display

Project 3: Shopping Cart System

- Array of items
- Add/remove items
- Show total count

Project 4: Menu System

- Food items array
- Display menu using loop

Day 20

JavaScript Objects + JSON

What is an Object in JavaScript?

An object is used to store **multiple related values as key-value pairs**.

Array stores values by index

Object stores values by name (key)

Simple Meaning:

Object = One real-world item with many details

Example:

A student has:

- name
- age
- course

Instead of separate variables:

```
let name = "Arun";
let age = 21;
let course = "JavaScript";
```

Use object:

```
let student = {
   name: "Arun",
   age: 21,
   course: "JavaScript"
};
```

Why Objects are Important?

Objects are used in almost every real project:

- User profiles
- Products
- Employee records
- API responses
- Database data

Object Syntax

```
let objectName = {
   key1: value1,
   key2: value2
};
```

Example:

```
let person = {
   name: "Alagu",
   city: "Tuticorin",
   age: 25
};
```

Accessing Object Values

There are 2 ways.

Dot Notation

```
console.log(person.name);
console.log(person.city);
```

Bracket Notation

```
console.log(person["age"]);
```

Output

Alagu
Tuticorin
25

Updating Object Values

```
person.age = 26;
console.log(person);
```

Adding New Property

```
person.job = "Trainer";
```

Deleting Property

```
delete person.city;
```

Object with Function (Method)

Objects can also contain functions.

```
let user = {
   name: "Arun",
   greet: function() {
      console.log("Hello");
   }
};
user.greet();
```

Loop Through Object

Use for...in

```
let student = {
   name: "Kumar",
   age: 22,
   course: "HTML"
};
for (let key in student) {
   console.log(key, student[key]);
}
```

Output:

name Kumar
age 22
course HTML

What is JSON?

JSON = JavaScript Object Notation
It is a text format used to exchange data between:

- Frontend

- Backend
- APIs
- Database

Simple Meaning:

JSON = Data format for sending/receiving information

JSON Structure

Looks like object, but keys are always in quotes.

```
let userData = {
   "name": "Ravi",
   "age": 24,
   "city": "Chennai"
};
```

Object vs JSON

Object	JSON
Used inside JS	Used for data transfer
Keys may be without quotes	Keys must be in quotes
Can store functions	Cannot store functions

Convert Object to JSON

Use JSON.stringify()

```
let person = {
   name: "Sam",
   age: 30
};

let jsonData = JSON.stringify(person);
console.log(jsonData);
```

Output:

```
{"name":"Sam","age":30}
```

Convert JSON to Object

Use JSON.parse()

```
let data = '{"name":"Sam","age":30}';

let obj = JSON.parse(data);

console.log(obj.name);
```

Output:

Sam

Real Project Use Case

In projects like:

- React
- Django
- APIs

JSON is used to send:

- product data
- user login details
- cart items
- employee records

Tasks (Day 20 Practice)

Task 1

Create object for student details

Task 2

Access object properties using dot notation

Task 3

Access using bracket notation

Task 4

Update object value

Task 5

Add new property

Task 6

Delete one property

Task 7

Loop object using for...in

Task 8

Convert object to JSON

Task 9

Convert JSON to object

Task 10

Create employee object and print all details

Mini Projects

Project 1: Student Profile System

Store:

- name

- age
- marks
- course

Project 2: Product Details System

Store:

- product name
- price
- category
- stock

Project 3: Employee Management Data

Store:

- employee id
- name
- department
- salary

Project 4: API Data Simulator

Create JSON data for:

- users
- products
- courses

Day 22

DOM Introduction (Document Object Model)

What is DOM?

DOM stands for **Document Object Model**.

It is the bridge between:

- HTML webpage
- JavaScript code

Using DOM, JavaScript can:

- access HTML elements
- change content
- change styles
- handle user actions

Simple Meaning

DOM = JavaScript controlling HTML page

Without DOM:

- JavaScript can run only in console

With DOM:

- JavaScript can change webpage directly

Real Example

Suppose your HTML has:

<h1>Hello</h1>

Using DOM, JavaScript can change it to:

<h1>Welcome</h1>

Automatically.

Why DOM is Important?

DOM is used for:

- button click actions
- form validation
- image sliders
- dynamic menus
- calculator apps
- todo apps
- login systems

Almost every interactive website uses DOM.

How DOM Works?

Browser converts HTML into tree structure.

Example:

```
<html>
  <body>
    <h1>Hello</h1>
    <p>Welcome</p>
  </body>
</html>
```

DOM Tree:

```
html
 └── body
     ├── h1
     └── p
```

Each tag becomes an object node.

JavaScript can access any node.

Selecting Elements in DOM

To work with HTML elements, first we select them.

Common methods:

1. getElementById()
2. getElementsByClassName()
3. getElementsByTagName()
4. querySelector()
5. querySelectorAll()

getElementById()

Most common method.

Used to select element by id.

Example

```
<h1 id="title">Hello</h1>
let heading = document.getElementById("title");

console.log(heading);
```

7. Changing Content

Use innerText

```
document.getElementById("title").innerText = "Welcome";
```

Output:

Before:

Hello

After:

Welcome

Changing HTML Content

Use innerHTML

```
<p id="msg"></p>
document.getElementById("msg").innerHTML = "<b>Bold Text</b>";
```

Difference

Property	Use
innerText	plain text
innerHTML	HTML tags allowed

Changing CSS using DOM

JavaScript can modify styles.

```
<h2 id="heading">JavaScript</h2>
let h = document.getElementById("heading");

h.style.color = "red";
h.style.backgroundColor = "yellow";
```

Result

Text color becomes red

Background becomes yellow

Full Example

```
<!DOCTYPE html>
<html>
<body>
<h1 id="title">Hello</h1>
<button onclick="changeText()">Click</button>
<script>
function changeText() {
   document.getElementById("title").innerText = "Welcome to DOM";
}
</script>
</body>
</html>
```

How It Works

- Page loads
- button appears
- when button clicked
- function runs
- text changes

DOM + JS Relationship

HTML	DOM	JavaScript
structure	object model	controls DOM

Real Project Usage

DOM is used in:

- React apps
- form validations
- calculators
- image galleries
- eCommerce cart
- quiz apps
- LMS projects

Tasks

Task 1

Create heading and change text using DOM

Task 2

Create paragraph and change content

Task 3

Change button text on click

Task 4

Change heading color

Task 5

Change background color of div

Task 6

Display username when button clicked

Task 7

Create welcome message updater

Task 8

Insert HTML using innerHTML

Task 9

Create color changer app

Task 10

Create simple dynamic profile card:

- name
- age
- city
- update using button click

Mini Projects

Project 1: Profile Viewer

HTML:

- name
- age
- button

JS:

- change content dynamically

Project 2: Theme Changer

Button click:

- change background
- change text color

Project 3: Message Display App

Input:

- user message

Output:

- show in page

Project 4: Product Info Display

Button click:

- show product details using DOM

Day 23

DOM Manipulation.

What is DOM Manipulation?

DOM Manipulation means:

- changing text
- changing styles
- adding elements
- removing elements
- modifying attributes
- updating content dynamically

Simple Meaning

DOM Manipulation = Changing webpage content using JavaScript

Why DOM Manipulation is Important?

Used in real websites for:

- showing login messages
- adding products to cart
- creating todo items
- updating form data
- image gallery changes
- popup creation
- dynamic dashboards

Common DOM Manipulation Operations

Main operations:

1. Change text
2. Change HTML
3. Change styles
4. Change attributes
5. Create elements
6. Append elements
7. Remove elements

Changing Text

Use innerText

```
<h1 id="title">Hello</h1>
document.getElementById("title").innerText = "Welcome";
```

Changing HTML Content

Use innerHTML

```
<div id="box"></div>
document.getElementById("box").innerHTML = "<h2>New Heading</h2>";
```

Difference

Method	Use
innerText	plain text
innerHTML	HTML tags

Changing Styles

Use .style

```
<p id="para">Text</p>
let p = document.getElementById("para");

p.style.color = "blue";
p.style.fontSize = "30px";
```

Changing Attributes

Attributes like:

- src
- href
- class
- id

Example

```
<img id="img" src="old.jpg">
document.getElementById("img").src = "new.jpg";
```

Creating New Elements

Use createElement()

```
let newPara = document.createElement("p");
newPara.innerText = "New paragraph created";
```

Adding Elements to Page

Use appendChild()

```
<div id="container"></div>
let p = document.createElement("p");
p.innerText = "Added dynamically";
```

```
document.getElementById("container").appendChild(p);
```

Removing Elements

Use remove()

```
<p id="removeMe">Delete me</p>
document.getElementById("removeMe").remove();
```

Full Example

```
<!DOCTYPE html>
<html>
<body>
<h1 id="title">Welcome</h1>
<button onclick="changePage()">Click</button>
<div id="box"></div>
<script>
function changePage() {
   let title = document.getElementById("title");
   title.innerText = "DOM Manipulated";
   title.style.color = "green";

   let p = document.createElement("p");
   p.innerText = "Paragraph added dynamically";

   document.getElementById("box").appendChild(p);
}
</script>
</body>
</html>
```

Real Use Cases

DOM manipulation is used in:

- React interfaces
- Todo apps
- Shopping carts
- LMS dashboards
- Form validation
- Admin panels

Tasks

Task 1

Change heading text

Task 2

Change paragraph color

Task 3

Change button text

Task 4

Update image source

Task 5

Create new paragraph using JS

Task 6

Append paragraph to div

Task 7

Create list item dynamically

Task 8

Remove one element

Task 9

Build simple dynamic card

Task 10

Create student details card:

- add
- update
- remove dynamically

Mini Projects

Project 1: Dynamic Profile Card

Display:

- name
- age
- city

Using JavaScript DOM.

Project 2: Notes App

- add note
- display note
- remove note

Project 3: Theme Switcher

Button click:

- change colors
- update heading

- append message

Project 4: Product Card Generator

Create product cards dynamically:

- product name
- price
- category

Day 24

JavaScript Event Handling

What is an Event?

An event is an action that happens in a webpage.

Examples:

- clicking a button
- typing in input
- moving mouse
- submitting a form
- pressing keyboard key

JavaScript can detect these actions and run code.

Simple Meaning

Event = User action on webpage

What is Event Handling?

Event Handling means:

Writing JavaScript code to respond when an event happens.

Example:

- click button → show message
- type input → display text
- submit form → validate

Why Event Handling is Important?

Used in real projects:

- login form
- signup form
- calculators
- cart buttons
- image slider
- search bars
- interactive dashboards

Common Events

Event	Trigger
click	button click
dblclick	double click
mouseover	mouse hover
mouseout	mouse leave
keyup	key released
keydown	key pressed
input	typing in input
submit	form submit

Click Event

Most basic event.

```
<button onclick="showMsg()">Click Me</button>
function showMsg() {
   alert("Button clicked");
}
```

Double Click Event

```
<button ondblclick="doubleClick()">Double Click</button>
function doubleClick() {
   alert("Double clicked");
}
```

Mouse Events

mouseover

```
<h1 onmouseover="hoverText()">Hover Here</h1>
function hoverText() {
   console.log("Mouse entered");
}
```

mouseout

```
<h1 onmouseout="leaveText()">Move Out</h1>
function leaveText() {
   console.log("Mouse left");
}
```

Keyboard Events

keyup

Runs when key is released.

```
<input type="text" onkeyup="showInput()">
```

```
function showInput() {
   console.log("Typing...");
}
```

Input Event

Used for live input detection.

```
<input type="text" id="name" oninput="displayName()">
<p id="result"></p>
function displayName() {
   let value = document.getElementById("name").value;
   document.getElementById("result").innerText = value;
}
```

Submit Event

Used in forms.

```
<form onsubmit="submitForm()">
   <input type="text">
   <button type="submit">Submit</button>
</form>
function submitForm() {
   alert("Form submitted");
}
```

addEventListener()

Professional way to handle events.

Instead of inline HTML event.

Syntax

```
element.addEventListener("event", function);
```

Example

```
<button id="btn">Click</button>
let btn = document.getElementById("btn");
btn.addEventListener("click", function() {
   alert("Clicked");
});
```

Why Use addEventListener?

Cleaner code

Used in modern projects

Preferred in React style logic

Supports multiple events

Full Example

```
<!DOCTYPE html>
<html>
<body>
<input type="text" id="username">
<button id="btn">Show</button>
<p id="output"></p>
<script>
let btn = document.getElementById("btn");
btn.addEventListener("click", function() {
   let name = document.getElementById("username").value;
   document.getElementById("output").innerText = name;
});
</script>
</body>
</html>
```

Tasks

Task 1

Create button click event

Task 2

Create double click event

Task 3

Create mouseover effect

Task 4

Create mouseout effect

Task 5

Show alert on button click

Task 6

Display input text live

Task 7

Change background color on click

Task 8

Create keyup event

Task 9

Use addEventListener for button

Task 10

Build live username preview system

Mini Projects

Project 1: Live Profile Preview

Input:

- name
- age

Display:

- update profile card live

Project 2: Feedback Form

- input
- submit
- show message

Project 3: Theme Changer

- click button
- change theme
- mouseover effect

Project 4: Product Search Box

- input typing
- show typed text live

Day 25: Form Validation

Form validation means checking whether the user has entered correct data before processing the form.

This is a very important topic because almost every website has forms:

- login
- signup
- contact
- feedback
- checkout
- registration

What is Form Validation?

Form validation is the process of checking user input.

It ensures:

- required fields are filled
- email format is correct
- password length is valid
- phone number format is correct

Simple Meaning

Validation = Checking input before submit

Why Validation is Important?

Without validation:

- empty form may submit
- wrong email can be entered
- weak password accepted
- bad user experience

With validation:

- accurate data
- better security
- better usability

Types of Validation

Two main types:

1. HTML validation
2. JavaScript validation

HTML Validation

Built-in browser validation using attributes.

Example:

```
<form>
   <input type="text" required>
   <input type="email" required>
  <button>Submit</button>
</form>
```

Common attributes

Attribute	Purpose
required	mandatory field
minlength	minimum length
maxlength	maximum length
pattern	custom pattern
type=email	email format

JavaScript Validation

Used for custom checking.

Example:

- password match
- custom username rules
- age check

Basic Example

```
<input type="text" id="name">
<button onclick="checkName()">Submit</button>
function checkName() {
   let name = document.getElementById("name").value;

   if (name === "") {
      alert("Name is required");
   } else {
      alert("Submitted");
   }
}
```

Validate Multiple Fields

```
<input type="text" id="username">
<input type="password" id="password">
<button onclick="validate()">Login</button>
function validate() {
   let user = document.getElementById("username").value;
   let pass = document.getElementById("password").value;

   if (user === "" || pass === "") {
      alert("All fields required");
   } else {
      alert("Login success");}
}
```

Email Validation

```
<input type="email" id="email">
<button onclick="checkEmail()">Submit</button>
function checkEmail() {
   let email = document.getElementById("email").value;

   if (email.includes("@")) {
      alert("Valid email");
   } else {
      alert("Invalid email");
   }
}
```

Password Length Validation

```
if (password.length < 6) {
   alert("Password must be at least 6 characters");
}
```

preventDefault()

When form submits, page refreshes automatically.

To stop that:

Use preventDefault()

Example

```
<form id="myForm">
   <input type="text" id="name">
   <button type="submit">Submit</button>
</form>
document.getElementById("myForm").addEventListener("submit",
function(event) {
   event.preventDefault();

   alert("Form checked");
});
```

Why preventDefault?

stops page reload

used in modern apps

required for JS validation

used in forms heavily

Real Project Use Cases

Used in:

- login pages
- signup systems
- LMS registration
- job portals
- eCommerce checkout
- employee systems

Frameworks like React and Django also rely on validation.

Tasks

Task 1

Validate empty name field

Task 2

Validate email input

Task 3

Validate password length

Task 4

Check age > 18

Task 5

Show error if phone number empty

Task 6

Create login validation

Task 7

Create signup form validation

Task 8

Use preventDefault()

Task 9

Show error messages below inputs

Task 10

Create complete registration validation:

- name
- email
- password
- confirm password

Mini Projects

Project 1: Login Form Validation

Fields:

- username
- password

Check:

- empty
- correct length

Project 2: Registration Form

Fields:

- name
- email
- phone
- password

Validate all.

Project 3: Student Admission Form

Fields:

- name
- age
- course

Validation:

- age > 18

Project 4: Checkout Form

Fields:

- address
- phone
- pincode

Validation:

- all required

Day 26

Fetch API + JSON

What is Fetch API?

The **Fetch API** is a JavaScript feature used to request data from servers or APIs.
It helps your webpage communicate with backend systems and retrieve live data.
Examples:

- getting products from server
- loading users
- fetching weather data
- sending form data
- reading API responses

Simple Meaning

Fetch API = JavaScript asking data from server

Why Fetch API is Important?

Modern websites use APIs for everything:

- eCommerce products
- LMS course list
- job portal jobs
- food delivery menus
- weather apps
- admin dashboards

Without Fetch API:

- dynamic websites are difficult

With Fetch API:

- live data can load automatically

What is an API?

API = Application Programming Interface

It acts like a bridge between frontend and backend.

Example:

Frontend (HTML + JS) requests

⬇

Backend (e.g. Django server)

⬇

Response comes in JSON format

What is JSON?

JSON = JavaScript Object Notation

It is the most common format for API data transfer.

Example JSON:

```
{
  "name": "Laptop",
  "price": 50000
}
```

Why JSON?

Because it is:

- lightweight
- easy to read
- easy for JavaScript
- used by all APIs

Basic Fetch Syntax

```
fetch("https://jsonplaceholder.typicode.com/users")
```

This sends request to API.

Full Fetch Example

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

Explanation

Step 1

Request data

fetch(...)

Step 2

Convert response to JSON

response.json()

Step 3

Use received data

console.log(data)

Sample API Response

```
[
  {
    "id": 1,
    "name": "Leanne Graham"
  },
  {
    "id": 2,
    "name": "Ervin Howell"
  }
]
```

This is an array of objects.

Display API Data

```
<ul id="users"></ul>
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => response.json())
    .then(data => {
        let output = "";

        data.forEach(user => {
            output += `<li>${user.name}</li>`;
        });
        document.getElementById("users").innerHTML = output;
    });
```

Why .then() is Used?

Fetching takes time.

JavaScript waits for response using promises.

.then() means:

After receiving data → run this code

Async / Await (Modern Way)

Cleaner syntax.

```
async function getUsers() {
   let response = await fetch("https://jsonplaceholder.typicode.com/users");
   let data = await response.json();
   console.log(data);
}
getUsers();
```

Difference

Method	Style
then()	older common
async/await	modern clean

Real Project Use Cases

Fetch API is used in:

- React apps
- eCommerce product loading
- LMS course loading
- Job portals
- Food delivery apps
- Admin dashboards
- Weather apps

Full Example Project

```
<!DOCTYPE html>
<html>
<body>
<button onclick="loadUsers()">Load Users</button>
<ul id="list"></ul>
<script>
function loadUsers() {
   fetch("https://jsonplaceholder.typicode.com/users")
     .then(res => res.json())
     .then(data => {
        let result = "";

        data.forEach(user => {
           result += `<li>${user.name}</li>`;
        });
```

```
        document.getElementById("list").innerHTML = result;
    });
}
</script>
</body>
</html>
```

Tasks

Task 1

Fetch users API and print in console

Task 2

Fetch posts API

Task 3

Display usernames in webpage

Task 4

Display emails only

Task 5

Display first 5 users

Task 6

Use async/await

Task 7

Show products from fake API

Task 8

Create user card list

Task 9

Display API data in table

Task 10

Create search filter for fetched users

Mini Projects

Project 1: User List App

Fetch:

- users API

Display:

- name
- email

Project 2: Post Viewer

Fetch:

- posts API

Display:

- title
- body

Project 3: Product Viewer

Fetch:

- fake store API

Display:

- image
- title
- price

Project 4: Weather App

Fetch weather API:

- city
- temperature
- condition

Day 27

API Integration Basics

What is API Integration?

API integration means connecting your frontend application to an external or backend API and using that data inside your webpage.

Simple flow:

Frontend (HTML/CSS/JavaScript)

⬇

API request

⬇

Server response (JSON)

⬇

Display data on webpage

Simple Meaning

API Integration = Connecting website with external data source

Why API Integration is Important?

Almost all modern web applications use APIs:

- React apps
- Django backend projects
- eCommerce product loading
- LMS course display
- job portals

- food delivery apps
- payment systems
- weather apps

Without API integration:

- only static data

With API integration:

- dynamic real-time data

Real Example

Suppose you have an eCommerce website.

Instead of manually writing product cards:

Phone

Laptop

API provides product data automatically.

JavaScript fetches it and shows dynamically.

Steps in API Integration

Basic steps:

1. API URL
2. Fetch request
3. Convert to JSON
4. Access data
5. Display on page

API URL

API endpoint is the link used to get data.

Example:

https://jsonplaceholder.typicode.com/users

This endpoint returns user data.

Basic Integration Example

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

How it Works?

fetch()

Sends request to API.

response.json()

Converts response into JavaScript object.

data

Actual API data received.

Display Data in HTML

```
<ul id="userList"></ul>
fetch("https://jsonplaceholder.typicode.com/users")
   .then(res => res.json())
   .then(users => {
      let output = "";

      users.forEach(user => {
         output += `<li>${user.name}</li>`;
      });

      document.getElementById("userList").innerHTML = output;
   });
```

API Response Structure

Most APIs return:

Array of objects

```
[
 {
   "id": 1,
   "name": "John"
 }
]
```

So you often combine:

- arrays
- objects
- loops
- DOM

Handling Errors

Sometimes API may fail.

Use catch().

```
fetch("https://jsonplaceholder.typicode.com/users")
   .then(res => res.json())
   .then(data => console.log(data))
   .catch(error => console.log(error));
```

Why catch?

To handle:

- network errors
- invalid URL
- server issues

POST Request Intro

GET → receive data

POST → send data

Basic example:

```
fetch("https://jsonplaceholder.typicode.com/posts", {
   method: "POST",
   body: JSON.stringify({
      title: "Hello"
   }),
   headers: {
      "Content-Type": "application/json"
   }
})
.then(res => res.json())
.then(data => console.log(data));
```

Real Project Use Cases

API integration used in:

- login systems
- product listing
- course management
- employee systems
- weather apps
- dashboards
- payment integration

Example with backend:

Frontend → JavaScript

Backend → Django API

Tasks

Task 1

Fetch users and print in console

Task 2

Display users in webpage

Task 3

Display emails only

Task 4

Display first 3 users

Task 5

Show API data in cards

Task 6

Use async/await API call

Task 7

Handle API error using catch

Task 8

Display posts API

Task 9

Create API search filter

Task 10

Create user profile viewer from API

Mini Projects

Project 1: User Directory

API:

- users

Display:

- name
- phone
- email

Project 2: Blog Post Viewer

API:

- posts

Display:

- title
- description

Project 3: Product App

API:

- fake store products

Display:

- image
- title
- price

Project 4: Weather Dashboard

API:

- city weather

Display:

- temperature
- humidity
- condition

Day 28

Mini Project – To-Do App

Day 28 combines the JavaScript topics you learned so far:

- Variables
- Conditions
- Loops
- Functions
- Arrays
- Objects
- DOM
- Events
- Form validation
- Local logic practice

A To-Do App is a good project because it uses all these together.

Project Goal

Build a task manager where user can:

- add task
- delete task
- mark completed
- display all tasks dynamically

Concepts Used

This project practices:

DOM manipulation
event handling
arrays
functions
loops
conditions

Project Structure

```
todo-app/
│
├── index.html
├── style.css
└── script.js
```

HTML Code

```
<!DOCTYPE html>
<html>
<head>
   <title>To Do App</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
   <h1>To-Do App</h
 <input type="text" id="taskInput" placeholder="Enter task">
   <button id="addBtn">Add Task</button>

   <ul id="taskList"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

CSS Code

```
body {
   font-family: Arial;
   background: #f2f2f2;
   display: flex;
   justify-content: center;
   margin-top: 50px;
}
.container {
   background: white;
   padding: 20px;
   width: 400px;
   border-radius: 10px;
   box-shadow: 0 0 10px gray;
```

```
}
input {
    width: 70%;
    padding: 10px;
}
button {
    padding: 10px;
}
li {
    margin-top: 10px;
    display: flex;
    justify-content: space-between;
}
```

JavaScript Code

```
let tasks = [];
let input = document.getElementById("taskInput");
let addBtn = document.getElementById("addBtn");
let taskList = document.getElementById("taskList");
addBtn.addEventListener("click", addTask);
function addTask() {
    let task = input.value.trim()
     if (task === "") {
        alert("Enter task");
        return;
    }
    tasks.push(task);
    input.value = "";
    displayTasks();
}
function displayTasks() {
    taskList.innerHTML = "";

    tasks.forEach((task, index) => {
        let li = document.createElement("li");

        li.innerHTML = `
            ${task}
```

```
            <button onclick="deleteTask(${index})">Delete</button>
        `;
        taskList.appendChild(li);
    });
}
function deleteTask(index) {
    tasks.splice(index, 1);
    displayTasks();
}
```

How It Works

Step 1

User types task

Step 2

Clicks Add button

Step 3

Task added to array

Step 4

DOM updates list

Step 5

Delete button removes task

Real Project Similarity

This concept is similar to:

- task manager apps
- note apps
- LMS assignment tracker
- employee daily updates
- shopping list apps

Tasks

Task 1

Add task

Task 2

Delete task

Task 3

Clear input after add

Task 4

Validate empty input

Task 5

Display tasks in list

Task 6

Add completed status

Task 7

Strike completed task

Task 8

Count total tasks

Task 9

Add date for task

Task 10

Store tasks in browser (advanced)

Day 29

React Introduction

What is React?

React is a **JavaScript library used for building user interfaces (UI)**.

It is mainly used for creating:

- websites
- web applications
- dashboards
- single page applications
- mobile apps (with React Native)

Simple Meaning

React = A tool to build modern interactive websites easily

Why React is Used?

Before React, developers used plain HTML + CSS + JavaScript.

Problem:

- large projects become hard to manage
- code repetition increases
- UI updates are difficult

React solves this by making UI:

reusable

faster

organized

component-based

Real Websites Using React

Many major companies use React:

- Facebook
- Instagram
- Netflix
- Airbnb
- WhatsApp web interface

What Makes React Special?

React introduces **component-based development**.

That means:

A webpage is split into small reusable parts.

Example:

A website has:

- Navbar
- Sidebar
- Cards
- Footer

Each can be built as separate component.

What is a Component?

Component = reusable UI block

Example:

A button component:

```
function Button() {
   return <button>Click</button>;
}
```

This can be reused many times.

Why Components?

Without components:

Same code repeated many times

With components:

Reuse same code anywhere

Example

One card component can display:

- products
- courses
- users
- employees

React Works with JSX

JSX = JavaScript XML

It allows writing HTML inside JavaScript.
Example:
const element = <h1>Hello React</h1>;
Looks like HTML but written inside JavaScript.

JSX Advantage

- easy to read
- easier UI creation
- combines logic + HTML

How React Works?

React updates only required parts of page.
Instead of reloading full page.
This makes apps faster.

Traditional Website

Click → page reload

React Website

Click → only specific content updates

Virtual DOM

One important concept:
React uses:

Virtual DOM

Virtual DOM = lightweight copy of actual webpage
It compares changes and updates only necessary parts.

Why?

Faster performance

React Project Structure

Usually React project has:

```
react-app/
│
├── public/
├── src/
│   ├── App.js
│   ├── index.js
│   └── components/
```

Important Files

App.js

Main component

index.js

Entry point

components/

Reusable UI components

First React Program

```
function App() {
   return (
      <h1>Hello React</h1>
   );
}
export default App;
```

Output

Shows:

Hello React

React vs JavaScript

JavaScript	React
Full programming language	UI library
manual DOM handling	automatic updates
more code	less code
hard for large apps	easy for large apps

React in Real Projects

React is used for:

- eCommerce
- LMS
- job portal
- food delivery
- admin dashboard
- employee system
- chat apps
- portfolio websites

Since you already built LMS, food delivery, and job portal projects, React is especially useful for making those UIs more modular and easier to scale.

Tasks

Task 1

Research what React is

Task 2

Install Node.js

Task 3

Create React project folder manually

Task 4

Identify App.js

Task 5

Write first Hello React program

Task 6

Create simple heading component

Task 7

Create button component

Task 8

Create paragraph component

Task 9

Combine 3 components

Task 10

Create simple profile page using components

Mini Projects

Project 1: Profile Card

Create component:

- name
- age
- city

Project 2: Product Card

Create:

- image
- title
- price

Project 3: Course Card

Create:

- course name
- trainer
- duration

Project 4: Employee Card

Create:

- employee name
- role
- department